Internamente, ddev identifica anche i volumi relativi al progetto (script volumes.sh) nell' esempio precedente avremo: venv.dev.python.foo. In pratica se il volume non esiste lo crea e lo abilita (permessi e ownership) all'uso nel container.

Perfetto: con volumi nominati per progetto, `venv.dev.python.foo`, il caso per **ricreare il container con `--rm` diventa ancora più forte**. Il container resta effimero, mentre lo stato costoso e riutilizzabile — virtualenv, dipendenze, cache — vive nel volume, il cui ciclo di vita è indipendente da quello del container.[1] [2]

Cosa cambia nel modello

Il tuo design separa correttamente tre livelli:

Livello	Esempio	Durata
Immagine	`dev.python`	Finché non la ricostruisci
Volume di progetto	`venv.dev.python.foo`	Finché non lo elimini esplicitamente
Container	`dev.python.foo`	Una sessione, con `--rm`

Con `--rm`, Docker elimina il container e il suo writable layer, ma **non** i volumi nominati montati al suo interno: `venv.dev.python.foo` sopravvive e può essere rimontato identico nel container successivo.[3] [1]

Quindi il flusso naturale diventa:

```
Ddev --target=Python/foo
  ├─ assicura immagine dev.python
  ├─ assicura volume venv.dev.python.foo
  ├─ imposta ownership/permessi del volume
  ├─ crea container dev.python.foo con --rm
  └─ all'uscita rimuove solo il container
```

Vantaggi concreti

- **Avvio successivo rapido**: il primo avvio installa dipendenze nel volume; quelli successivi riutilizzano `venv`, `node_modules`, cache di compilazione, ecc.
- **Ambiente pulito a ogni sessione**: pacchetti installati manualmente, file temporanei, cronologie shell, processi e modifiche al filesystem interno non sopravvivono.
- **Nessun container orfano**: non devi gestire container fermi né distinguere tra "riavvia" e "ricrea con nuova configurazione".
- **Cache consapevolmente persistente**: la persistenza è limitata agli artefatti che hai dichiarato come volumi, non è un effetto collaterale del container.
- **Coerenza con l'aggiornamento dell'immagine**: ricreando il container usi la versione corrente di `dev.python`; il volume conserva solo i dati di progetto, non una vecchia configurazione del container.

Attenzioni su ownership e permessi

Il punto più delicato è proprio la fase in cui `volumes.sh` imposta ownership e permessi.

- Un volume nominato, al primo montaggio su un percorso esistente nell'immagine e se vuoto, può ereditare contenuto, ownership e permessi da quel percorso dell'immagine. Dopo il primo uso, Docker non riapplica automaticamente l'ownership dell'immagine. [4]
- Se il container lavora come utente non-root, l'UID/GID del processo nel container deve corrispondere all'owner effettivo del contenuto del volume; altrimenti otterrai errori di scrittura o file creati come root. [5] [4]
- Se cambi l'UID/GID dell'utente di sviluppo, il nome dell'immagine o la struttura del volume, il volume esistente può conservare permessi obsoleti. In quel caso serve una migrazione o un reset esplicito.

Una convenzione robusta sarebbe:

```
venv.<image>.<project>.<uid>-<gid>
```

Per esempio:

```
venv.dev.python.foo.1000-1000
```

Così un cambio di UID/GID non provoca un volume illeggibile, ma crea un nuovo volume coerente. Il vecchio resta disponibile per eventuale migrazione o pulizia.

Cosa NON mettere nel volume

Il volume deve contenere **artefatti rigenerabili o dipendenze**, non stato essenziale del progetto:

- Adatto: `venv/`, `node_modules/`, `target/`, `.cargo/registry`, `.m2/`, `.gradle/`, cache di tool.
- Da evitare: sorgenti, test, configurazioni versionate, credenziali, database non volatili, log che vuoi conservare, output che costituiscono un risultato finale.

Per sorgenti e file di progetto continua a usare un bind mount della directory host: è la fonte di verità, versionata e modificabile dall'editor. I volumi sono invece uno strato di cache/ambiente. [6] [7]

Gestione del volume

Ti converrebbe aggiungere a `Ddev` comandi espliciti per il ciclo di vita del volume:

```
Ddev --target=Python/foo --rebuild-image
Ddev --target=Python/foo --reset-volume
Ddev --target=Python/foo --prune-volumes
Ddev --target=Python/foo --inspect-volume
```

In particolare:

- `--reset-volume` dovrebbe rimuovere `venv.dev.python.foo` e ricrearlo con permessi corretti; è utile dopo cambi di immagine, UID/GID o versioni principali delle dipendenze.
- `--prune-volumes` dovrebbe eliminare solo volumi il cui progetto non esiste più, evitando cancellazioni per progetti spostati o rinominati.
- L'hash della configurazione — immagine, UID/GID, percorso di mount, variabili rilevanti — può essere incluso nel nome del volume o salvato come label Docker. Così puoi rilevare automaticamente un volume incompatibile.

Raccomandazione

Mantieni `--rm -it` come default. Con volumi nominati gestiti da `volumes.sh`, ottieni il meglio dei due mondi: **container monouso e riproducibile**, ma **venv,** `node_modules` **e cache persistenti**. Usa stop/start solo per un container "servizio" esplicito — database, dev server, watcher — non per la normale sessione di sviluppo. [2] [1]

⁂

1. https://docs.docker.com/engine/storage/volumes/
2. https://www.portainer.io/blog/persistent-storage-docker-bind-mounts-and-named-volumes
3. https://apxml.com/courses/docker-for-ml-projects/chapter-3-managing-ml-data-containers/docker-volumes-persistence
4. https://www.dash0.com/faq/how-to-manage-permissions-for-docker-shared-volumes
5. https://eastondev.com/blog/en/posts/dev/20251217-docker-mount-permissions-guide/
6. https://docs.docker.com/engine/storage/bind-mounts/
7. https://semaphore.io/blog/docker-volumes
8. https://dev.to/i_am_vesh/docker-volumes-and-persistence-a-comprehensive-guide-docker-day-3-1e1p
9. https://www.geeksforgeeks.org/devops/what-is-docker-volume/
10. https://agenda.infn.it/event/36241/sessions/26091/attachments/109819/156106/Data management in Docker.pdf
11. https://forums.docker.com/t/bind-mount-with-current-users-ownership/147176

12. https://github.com/docker/for-mac/issues/6734
13. https://blog.stackademic.com/docker-volumes-explained-the-complete-guide-to-persistent-storage-2025-5a6e1c9560d3
14. https://forums.docker.com/t/how-to-mount-a-docker-volume-so-as-writeable-by-a-non-root-user-within-the-container/144321
15. https://devops.stackexchange.com/questions/12713/docker-mounts-files-as-uid-1001